at index `0` and then the bit at index `3`, or vice versa; we can thus consider sets instead of sequences.

All considered, we only (at least in theory) allow a fraction of all the possible sets of exactly `k` (distinct) indices drawn from the range `0..m-1`, and the number of all these possible (valid) sets is given by:

$$\binom{m}{k} = \frac{m!}{k! \cdot (m-k)!}$$

The binomial coefficient[17] in the formula above expresses the number of ways in which we can extract `k` (unique) elements from a set of size `m`, and hence it tells us how many sets with exactly `k` distinct indices our `k` hash functions can return.

If we want each key to map to a different set of `k` indices, then given `k` and `n`, the number of keys to store, we can compute a lower bound for `m`, the size of the buffer, by using the formula above.

Another way to look at the question is that, given with a sequence of `m` bits (the buffer), we can only represent 2^m different values, and so the Bloom filter can only be, at a given time, in one of 2^m possible states: this, however, only gives us a loose (although easier to compute) bound on `n`, because we would not take into consideration that we will store `k` bits per key (2^m becomes an exact bound only for `k==1`).